Lecture 10 Scribe

L10_S2.pdf
PDF

> Generate a lecture scribe intended to serve as reference material for exam preparation. Use *ONLY* the provided context (Lecture 10 slides).
> *Instructions:*
> 1. *Purpose:* The scribe must be exam-ready, strictly following the content of the slides.
> 2. *Content:* Include definitions, assumptions, theorem statements, proofs or derivations, and worked examples *exactly* as presented in the lecture materials.
> 3. *Constraints:* Do not introduce new examples, outside information, or hallucinations. Do not use phrases like 'for example' to add your own content.
> 4. *Structure:* Organize the scribe clearly with appropriate headings corresponding to the lecture flow.
>
>
>
> *Goal:* Produce a clean, accurate revision document based solely on the attached file."
>
>

Lecture 10 Scribe

Course: CSE400 – Fundamentals of Probability in Computing

Topic: Randomized Min-Cut Algorithm

Source: Lecture 10 Slides

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The min-cut algorithm is used in applications related to **network connectivity, reliability, and optimization**.

Applications listed in the lecture slides:

- **Network Design:** Improves communication efficiency and optimizes network flow by finding the minimum capacity cut.
- **Communication Networks:** Helps understand network vulnerability to failures and supports building robust and fault-tolerant networks.
- **VLSI Design:** Useful for partitioning circuits into smaller components and reducing interconnectivity complexity.

1.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components.

Definition: Min-Cut Problem

Given a graph $G = (V, E)$ with n vertices, the **minimum cut (min-cut)** problem is to find a minimum cardinality cut-set in G.

Min-cut algorithms such as Karger's algorithm are random and can be sensitive to the initial choice of edges. If critical edges are contracted early, the algorithm might find a smaller cut.

1.3 Edge Contraction Operation

The main operation in the algorithm is **edge contraction**, which removes an edge from a graph while merging the two vertices.

When contracting an edge (u, v):

- Merge vertices u and v into one vertex.
- Eliminate all edges connecting u and v.
- Retain all other edges.
- The new graph may have parallel edges but no self-loops.

2. Successful and Unsuccessful Runs

2.1 Successful Min-Cut Run

A **successful min-cut run** refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

2.2 Unsuccessful Min-Cut Run

An **unsuccessful min-cut run** refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

---

3. Max-Flow Min-Cut Theorem

Theorem Statement

"In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut."

Definitions

- **Capacity of a cut:** Sum of capacities of edges in the cut oriented from a vertex in X to a vertex in Y.
- **Minimum cut:** Cut with the smallest possible capacity.
- **Minimum cut capacity:** Capacity of the minimum cut.
- **Maximum flow:** Largest possible flow from source S to sink T.

---

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Theorem

Let s and t be two vertices of a graph G. Let $G/\{s, t\}$ be the graph obtained by merging s and t.
A minimum cut of G can be obtained by taking the smaller of:

- A minimum s-t cut of G, and
- A minimum cut of $G/\{s, t\}$.

This holds because:

- Either a minimum cut separates s and t, or

- It does not, in which case the minimum cut of the merged graph gives the result.

4.2 Deterministic Min-Cut Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

- $A \leftarrow \{a\}$
- While $A \neq V$:
  - Add to A the most tightly connected vertex
- Return the cut weight as the cut of the phase.

Algorithm 2: MinimumCut(G)

- While $|V| \geqslant 1$:
  - Choose any $a \in V$
  - Run MinimumCutPhase(G, a)
  - If cut-of-phase lighter than current minimum cut:
    - Store it
  - Shrink G by merging the last two vertices added
- Return minimum cut.

---

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success and can give a more accurate estimate of the minimum cut with fewer iterations.

Reasons listed:

- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Heuristic nature
- Robustness

5.2 Karger's Randomized Algorithm

The algorithm repeatedly contracts randomly chosen edges until only two vertices remain, producing a cut.

5.3 Randomized Min-Cut Pseudocode

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input: Undirected multigraph G with n vertices and integer constant $\alpha > 0$.
Output: A cut C of G.

- If $n \leqslant \alpha^3$:
  - $C \leftarrow$ a min-cut of G using brute force search
- Else:
  - For $i = 1$ to α:
    - $G' \leftarrow$ multigraph obtained by applying $n - \frac{n}{\sqrt{\alpha}}$ random contractions
    - $C' \leftarrow$ Recursive-Randomized-Min-Cut(G', α)
    - If $i = 1$ or $|C'| < |C|$:
      - $C \leftarrow C'$
- Return C.

---

6. Comparison: Deterministic vs Randomized Min-Cut

Deterministic Min-Cut

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer–Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

where V is vertices and E is edges.

Randomized Min-Cut

- Provides approximate minimum cut with high probability.
- Karger's algorithm time complexity:

$$O(V^2)$$

where V is number of vertices.

---

7. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least

$$\frac{2}{n(n-1)}.$$

---

8. Python Simulation

Students are instructed to open the Campuswire post for Lecture 10 and download the provided `.ipynb` file for simulation.

End of Lecture 10 Scribe